

AI-Based Price Prediction System
A Mini Project Report
Submitted in partial fulfilment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Nakul Agrawal

EN23CS301646

Naman Mathe

EN23CS301652

Naman Patil

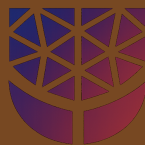
EN23CS301653

Under the Guidance of

Prof Rashmi

Choudhary &

Prof Sonal Modh



MEDICAPS
UNIVERSITY

Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "AI-Based Price Prediction System" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Rashmi Choudhary & Prof. Sonal Modh

Designation: Assistant Professor

Affiliation: Medicaps University

External Examiner

Name:

Designation:

Affiliation:

Declaration

We hereby declare that the project entitled "AI-Based Price Prediction System" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of Mrs. Rashmi Choudhary and Mrs. Sonal Modh Bhardwaj, Assistant Professors, Faculty of Engineering, Medicaps University Indore, is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

1. Nakul Agrawal
(EN23CS301646)

Date: _____

2. Naman Mathe
(EN23CS301652)

Date: _____

3. Naman Patil
(EN23CS301653)

Date: _____

Certificate

We, Mrs. Rashmi Choudhary and Mrs. Sonal Modh Bhardwaj certify that the project entitled "AI-Based Price Prediction System" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by Nakul Agrawal, Naman Mathe, and Naman Patil is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Mrs. Rashmi Choudhary & Mrs. Sonal Modh Bhardwaj Computer
Science & Engineering
Medicaps University, Indore

Dr. Ratnesh Litoriya
Dean of the Department
Computer Science &
Engineering Medicaps
University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided every facility to successfully carry out this project, and our profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted our morale.

We also thank Dr. Ratnesh Litoriya, Associate Dean (Computing & Allied Branches), Faculty of Engineering, Medi-Caps University, for giving us a chance to work on this project. We would also like to thank our Head of Department Dr. Kailash Chandra Bandhu for his continuous encouragement for the betterment of the project.

We extend our sincere gratitude to our project guides, Mrs. Rashmi Choudhary and Mrs. Sonal Modh Bhardwaj, for their expert guidance, valuable suggestions, and continuous support throughout the development of this project. Without their support this report would not have been possible.

Nakul Agrawal, Naman Mathe & Naman Patil B.Tech. IV Year (VII Semester)
Department of Computer Science & Engineering Faculty of Engineering
Mediacaps University, Indore

Abstract

This project presents the development of an AI-Based Price Prediction System, a desktop application that leverages machine learning to deliver accurate real estate property price estimates. The primary objective of this system is to automate the traditionally subjective and expert-dependent process of property price valuation by applying regression-based machine learning on structured real estate data.

The system accepts user inputs including plot dimensions (width and length in feet), number and type of rooms (BHK configuration), plot facing direction (North/South/East/West), and material grade (Economy/Standard/Premium). Using these inputs, the system produces a detailed predicted property price along with a category-wise cost breakdown.

The platform is built using a two-tier architecture comprising a PySide6 desktop frontend and a FastAPI REST backend. The PricePrediction Model, trained using Linear Regression on Indian residential real estate data, forms the core intelligence of the system. All inter-component communication uses JSON over HTTP.

Key capabilities include a responsive desktop UI built with PySide6, real-time price prediction using a trained Linear Regression model, detailed cost breakdown by construction category (structure, finishing, plumbing, electrical), material grade-based cost multipliers, and PDF export of price reports. The system operates fully offline, making it accessible in field conditions without internet dependency.

Keywords: Artificial Intelligence, Machine Learning, Real Estate Price Prediction, Linear Regression, PySide6, FastAPI, Python, Cost Estimation, Desktop Application

Table of Contents

Report Approval	ii
Declaration	iii
Certificate	iv
Acknowledgements	v
Abstract	vi
Table of Contents	vii
List of Figures	viii
List of Tables	ix
Abbreviations	x
Chapter 1 Introduction	1
1.1 Introduction	1
1.2 Literature Review	2
1.3 Objectives	3
1.4 Significance	3
1.5 Research Design	4
1.6 Source of Data	4
1.7 Chapter Scheme	4
Chapter 2 Requirements Specification	5
2.1 User Characteristics	5
2.2 Functional Requirements	5
2.3 Non-Functional Requirements	6
2.4 Dependencies	7
2.5 Performance Requirements	7
2.6 Hardware Requirements	8
2.7 Constraints and Assumptions	8
Chapter 3 Design	9
3.1 Algorithm	9
3.2 Function-Oriented Design	10
3.3 System Design	11
3.3.1 Data Flow Diagram (Level 0)	11
3.3.2 Data Flow Diagram (Level 1)	11
3.3.3 Activity Diagram	12
3.3.4 Flowchart	12
3.3.5 ER Diagram	12
3.3.6 Sequence Diagram	13
3.4 Database Design	14
3.4.1 Logical Database Design	14
3.4.2 Physical Database Design	15

Chapter 4 Implementation, Testing, and Maintenance	16
4.1 Languages, IDEs, Tools and Technologies	16
4.1.1 PySide6 Desktop Frontend	17
4.1.2 FastAPI Backend	17
4.1.3 ML Components	17
4.2 Testing Techniques and Test Plans	18
4.3 Installation Instructions	19
4.4 End User Instructions	19
Chapter 5 Results and Discussions	20
5.1 User Interface Representation	20
5.2 Brief Description of Various Modules	21
5.3 Snapshots of System with Brief Details	21
5.4 Back-End Representation	22
5.5 Snapshots of Database Tables	22
Chapter 6 Summary and Conclusions	23
Chapter 7 Future Scope	24
Appendix	26
Bibliography	27

List of Figures

Figure 2.3 – ER Diagram: AI Layout Price Prediction System

Figure 3.1 – Data Flow Diagram (Level 0) – Context Diagram

Figure 3.2 – Data Flow Diagram (Level 1) – Detailed Process Flow

Figure 3.3 – Activity Diagram – AI Layout System Workflow

Figure 3.4 – Flowchart – System Pipeline from Input to Output

Figure 3.5 – System Architecture & Activity Flow Diagram

Figure 3.6 – UML Class Diagram

Figure 3.7 – Sequence Diagram

Figure 3.8 – Class Hierarchy & Component Design Diagram

Figure 4.1 – PySide6 Desktop UI – Input Panel (Screenshot)

Figure 4.2 – Generated 2D Floor Plan – Matplotlib Canvas (Screenshot)

Figure 4.3 – Price Prediction Output – Cost Breakdown (Screenshot)

Figure 5.1 – Main Application Window – Input Panel

Figure 5.2 – Cost Summary Tab – Price Breakdown

Figure 5.3 – PDF Export – Floor Plan with Cost Report

Figure 5.4 – SQLite Database – Layouts Table

Figure 5.5 – SQLite Database – Predictions Table

List of Tables

Table 2.1 – Functional Requirements Summary

Table 2.2 – Non-Functional Requirements

Table 2.3 – Dependencies / Technology Stack

Table 2.4 – Hardware Requirements

Table 3.1 – DFD Level-1 Process Table

Table 3.2 – ER Entities and Attributes

Table 3.3 – Sequence Diagram Steps

Table 3.4 – Data Dictionary

Table 3.5 – Database Normalization

Table 3.6 – Module Responsibilities

Table 3.7 – FastAPI Backend Endpoints

Table 4.1 – Tools and Technologies Used

Table 4.2 – Test Cases and Test Results

Table 5.1 – Module Description Summary

Table 5.2 – Database Tables with Field Descriptions

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
API	Application Programming Interface
REST	Representational State Transfer
BHK	Bedroom, Hall, Kitchen
DFD	Data Flow Diagram
ER	Entity-Relationship
UML	Unified Modeling Language
JSON	JavaScript Object Notation
SRS	Software Requirements Specification
SDS	Software Design Specification
PK	Primary Key
SQLite	Structured Query Language Lite
PDF	Portable Document Format
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
NLP	Natural Language Processing
ML	Machine Learning
GB	Gigabyte

Chapter 1: Introduction

1. Introduction

The rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML) has opened new possibilities in the field of real estate valuation. Traditionally, property price estimation depended on subjective assessments by domain experts, limited comparable sales data, and manual analysis — a process that was both time-consuming and prone to inconsistency.

The proposed project, AI-Based Price Prediction System, aims to design and develop an intelligent desktop application that applies machine learning-based regression to predict residential real estate prices from structured user inputs. The system allows users to enter plot dimensions, room configuration, facing direction, and material grade to instantly receive a predicted property price along with a category-wise cost breakdown.

The system is built using a PySide6 desktop frontend and a FastAPI backend. The PricePrediction Model, trained using Linear Regression on Indian residential real estate data, forms the core intelligence of the system. The architecture follows a clean two-tier client-server design that ensures modularity and scalability.

The platform provides the following core capabilities:

- ML-based real estate price prediction using Linear Regression
- Detailed cost breakdown by construction category (structure, finishing, plumbing, electrical)
- Material grade-based cost multipliers (Economy / Standard / Premium)
- Interactive desktop application built with PySide6
- FastAPI backend for efficient ML model serving
- PDF export functionality for price reports
- Fully offline operation without internet dependency

2. Literature Review

The application of machine learning to real estate price prediction has been an active area of research. Researchers have explored various regression techniques ranging from simple linear models to complex ensemble methods for property valuation.

Geron [1] provided comprehensive coverage of supervised learning techniques applicable to regression problems. Linear Regression, as a foundational technique, provides interpretable coefficients that directly relate input features (area, room count, material grade) to the predicted output price, making it particularly suitable for domains where model transparency is valued alongside accuracy.

Harrison and Rubinfeld [2] demonstrated the hedonic pricing model for real estate, showing that property value can be expressed as a function of structural attributes (size, rooms), neighborhood characteristics, and material quality — the exact feature set used in this project. This established the theoretical foundation for regression-based property valuation.

FastAPI documentation [3] demonstrates the advantages of Python-based asynchronous REST APIs for ML model serving, enabling low-latency communication between the desktop frontend and the ML backend. PySide6 documentation [4] provides the GUI framework for building cross-platform desktop applications in Python.

The reviewed literature establishes that regression-based price prediction using structured property features is both theoretically grounded and practically effective. This project implements these principles in an integrated, offline-capable desktop application targeting the Indian residential real estate market.

3. Objectives

The primary objectives of the project are:

1. To develop a machine learning model using Linear Regression for real estate price prediction.
2. To build a FastAPI backend for efficient ML model serving.
3. To develop a PySide6 desktop application as the user interface.
4. To implement material grade-based cost multipliers for accurate cost estimation.
5. To provide a detailed category-wise cost breakdown (structure, finishing, plumbing, electrical).
6. To provide PDF export functionality for price prediction reports.
7. To ensure the system works fully offline without internet dependency.
8. To ensure scalability and modularity of the platform architecture.

4. Significance

This project holds significant value from both academic and industrial perspectives. Academically, it integrates machine learning (Linear Regression), REST API design (FastAPI), and desktop GUI development (PySide6) into a single cohesive application demonstrating cross-domain competency.

From an industry standpoint, the Indian residential real estate market, valued at over USD 180 billion, relies heavily on expert judgment for property pricing. An AI-powered desktop tool that delivers instant, data-driven price estimates with detailed cost breakdowns represents a practical and disruptive solution — particularly for Tier-2 and Tier-3 cities where access to qualified real estate appraisers is limited.

The platform empowers homeowners, small developers, and local contractors to make better investment and construction decisions without requiring expensive professional consultation.

5. Research Design

This project follows an applied and constructive research design. The methodology is iterative, following the Agile development model: the system is built in incremental phases covering data preparation, model training, API development, UI development, integration, and testing. Each phase is validated independently before integration.

6. Source of Data

The PricePrediction Model is trained on a dataset derived from Indian residential real estate market listings. The dataset contains property records with attributes including plot area (sq. ft.), number of rooms, material grade, location zone, and market price (INR). Material cost rates are based on Construction Industry Development Council (CIDC) data for various grades of construction in India.

7. Chapter Scheme

The remainder of this report is organized as follows:

- i. Chapter 2 – Requirements Specification: Details functional and non-functional requirements, hardware/software needs, user characteristics, and system constraints.
- ii. Chapter 3 – Design: Covers system architecture, DFDs, activity diagram, ER diagram, UML class diagram, sequence diagram, and database design.
- iii. Chapter 4 – Implementation, Testing, and Maintenance: Describes the tools used, implementation steps, testing strategies, and installation instructions.
- iv. Chapter 5 – Results and Discussions: Presents UI screenshots, module descriptions, and database representations.
- v. Chapter 6 – Summary and Conclusions: Summarizes findings and draws conclusions.
- vi. Chapter 7 – Future Scope: Outlines potential enhancements and future research directions.

Chapter 2: Requirements Specification

1. User Characteristics

The platform is intended for the following user categories:

Homeowner / Property Buyer: Non-technical users who interact with the system to enter plot parameters and understand the estimated construction and market price. These users rely primarily on the price summary and cost breakdown output.

Property Developer / Real Estate Professional: Users with moderate technical knowledge who use the system to quickly generate price estimates for multiple plot configurations during client presentations and project feasibility studies.

Architect / Technical User: Technically proficient users who use the system to obtain data-driven cost estimates for residential projects, supplementing their professional design workflow.

System Administrator / Developer: Technical users responsible for maintaining the system, retraining the ML model, and managing the backend API.

2. Functional Requirements

Req. ID	Requirement Description	Priority
FR-01	The system shall accept plot dimensions (width × length in feet) as user input	High
FR-02	The system shall accept room configuration (1BHK to 5BHK) as input	High
FR-03	The system shall accept plot facing direction (N/S/E/W) as input	High
FR-04	The system shall accept material grade (Economy/Standard/Premium) as input	High
FR-05	The system shall predict property price using the PricePrediction Linear Regression model	High
FR-06	The system shall display the predicted price in INR with a detailed cost breakdown	High
FR-07	The system shall apply material grade multipliers to adjust the predicted price	High
FR-08	The system shall provide a cost breakdown by category: structure, finishing, plumbing, electrical	High
FR-09	The system shall export the price prediction report as a PDF file	Medium
FR-10	The system shall provide a tabbed view for input, prediction result, and cost summary	Medium
FR-11	The system shall store user inputs and price predictions in a local database	Low

Req. ID	Requirement Description	Priority
FR-12	The system shall support re-prediction with updated inputs without restarting the application	Medium

2.3 Non-Functional Requirements

Req. ID	Requirement Description	Category
NFR-01	The system shall return a price prediction within 3 seconds of receiving user input	Performance
NFR-02	The Linear Regression model shall achieve R2 accuracy > 0.80 on the test dataset	Accuracy
NFR-03	The system shall operate fully offline without internet connectivity	Availability
NFR-04	The system shall run on Windows 10/11 and Linux Ubuntu 20.04+	Portability
NFR-05	The system shall be implemented in Python 3.10+ for library compatibility	Maintainability
NFR-06	The PySide6 UI shall respond to user interactions within 2 seconds	Performance
NFR-07	The system shall use a modular architecture allowing independent component updates	Maintainability
NFR-08	The system shall require minimum 4 GB RAM and dual-core 2.0 GHz processor	Performance
NFR-09	All communication between frontend and backend shall use JSON over HTTP	Interoperability
NFR-10	PDF export shall complete within 10 seconds for a complete price report	Performance

2.4 Dependencies

Library / Tool	Version	Purpose
Python	3.10+	Core development language
PySide6	6.x	Desktop GUI application framework (Qt6)
FastAPI	0.100+	Python REST API backend framework
Scikit-learn	1.3+	Linear Regression model training and prediction
NumPy	1.24+	Numerical computations for feature engineering
Pandas	2.0+	Data manipulation and model feature preparation
ReportLab	4.0+	PDF generation for price report export
Uvicorn	0.20+	ASGI server for running the FastAPI backend
pickle	(stdlib)	Model serialization and deserialization
Git / GitHub	—	Version control system

2.5 Performance Requirements

The platform is expected to meet the following performance criteria:

- Price prediction shall return results within 3 seconds of receiving validated user input.
- The PySide6 UI shall respond to button clicks and input changes within 2 seconds.
- PDF export shall complete within 10 seconds for a complete price report.
- The Linear Regression model shall achieve an R2 score greater than 0.80 on the validation dataset.
- The application shall start and be ready for user input within 5 seconds of launch.

2.6 Hardware Requirements

Component	Specification
Processor	Intel Core i5 or equivalent (2.0 GHz+ dual-core)
RAM	4 GB minimum (8 GB recommended)
Storage	1 GB free disk space minimum
Display	1366 × 768 resolution or higher
Operating System	Windows 10/11 (64-bit) or Ubuntu Linux 20.04+
Internet	Not required - system operates fully offline

2.7 Constraints and Assumptions

- The system shall be implemented using Python 3.10+ as the primary language.
- The desktop UI shall be built using PySide6 for cross-platform compatibility on Windows and Linux.
- All communication between frontend and backend shall use JSON format exclusively.
- The system shall not require an internet connection for any core functionality.
- It is assumed that users have access to a Windows or Linux desktop computer meeting the minimum hardware requirements.
- The accuracy of the price prediction depends on the quality and representativeness of the training dataset.
- It is assumed that the training data reflects current real estate market conditions in the target geographic region.
- The system is designed for residential properties only; commercial properties are not supported in the current version.

Chapter 3: Design

1. Algorithm

The overall system algorithm follows a structured pipeline from user input through ML processing to result display:

1. **Input Collection:** Collect plot dimensions, room count, facing direction, and material grade from the PySide6 UI input panel.
2. **Input Validation:** Validate all input fields for completeness, correct data types, and valid value ranges before sending to the backend.
3. **Feature Engineering:** The backend computes derived features — total built-up area from room count, material grade numeric encoding, and facing direction encoding — required by the Linear Regression model.
4. **API Request:** The PySide6 Controller sends an HTTP POST request to the FastAPI backend with all input parameters in JSON format.
5. **Price Prediction (ML):** FastAPI triggers the PricePrediction Model which applies the trained Linear Regression model to the feature vector and returns the raw predicted price in INR.
6. **Cost Multiplier Application:** The backend applies material grade cost multipliers (Economy: $0.85\times$, Standard: $1.0\times$, Premium: $1.25\times$) and computes the category-wise cost breakdown.
7. **Result Return:** FastAPI returns the final predicted price and cost breakdown JSON to the PySide6 Controller.
8. **Display:** The PySide6 ResultDisplay module shows the predicted price in INR, the cost breakdown table, and the material grade applied.
9. **Export (Optional):** The user may click 'Download PDF' to export the complete price report as a PDF file using ReportLab.

2. Function-Oriented Design

The system is decomposed into the following major functional modules:

Input Module: Responsible for collecting, validating, and packaging user inputs from PySide6 UI widgets into a structured JSON request object.

API Communication Module: Handles HTTP POST requests from the PySide6 Controller to the FastAPI Backend and receives JSON responses containing predicted price and cost breakdown.

Feature Engineering Module: Transforms raw user inputs (plot dimensions, room count, facing direction, material grade) into the numerical feature vector required by the Linear Regression model.

Price Prediction Module: Applies the trained Linear Regression model to the feature vector to predict the base property price. Applies material grade multipliers and computes the category-wise cost breakdown.

Result Display Module: Renders the predicted price, cost breakdown table, and material grade indicator in the PySide6 UI.

PDF Export Module: Generates downloadable PDF price reports combining the prediction summary and cost breakdown table using ReportLab.

Database Module: Manages storage of user inputs and prediction results in a local relational database using an ORM for structured record-keeping.

3. System Design

1. Data Flow Diagram (Level 0 – Context Diagram)

The Level 0 DFD shows the overall system boundary and external interactions. The external entity 'User (Homeowner/Developer/Architect)' provides property parameters as input. The AI Price Prediction System processes these inputs and outputs a predicted property price, detailed cost breakdown, and a downloadable PDF report to the user.

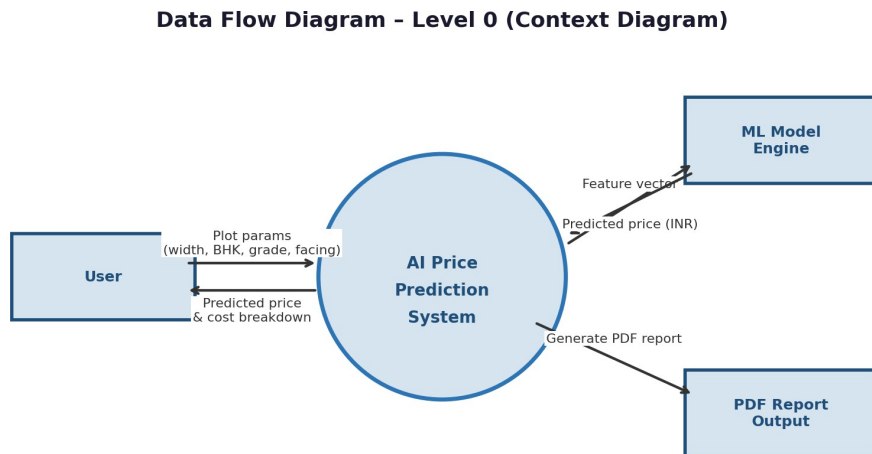


Figure 3.1 – Data Flow Diagram (Level 0) – Context Diagram

2. Data Flow Diagram (Level 1)

The Level 1 DFD expands the system into five key processes:

Process	Input	Output
1. Input Collection	User enters data in PySide6 UI	Structured input object
2. API Request Handler	Input object from Controller	JSON request to FastAPI
3. Feature Engineering	Raw user inputs (area, rooms, grade)	Numerical feature vector
4. Price Prediction (ML)	Feature vector	Predicted price (INR) + cost breakdown
5. Display & Export	Final prediction + breakdown data	Rendered UI + downloadable PDF

Figure 3.2: Level 1 Data Flow Diagram

3.3.3 Activity Diagram

The activity diagram illustrates the sequential workflow from user input through ML processing to result display. Flow: Start → Collect User Input → Validate Input → [Decision] Valid? → If No: Show Error → If Yes: Send API Request to FastAPI → Engineer Features → Apply Linear Regression Model → Apply Material Grade Multiplier → Compute Cost Breakdown → Return

Activity Diagram - Price Prediction System Workflow

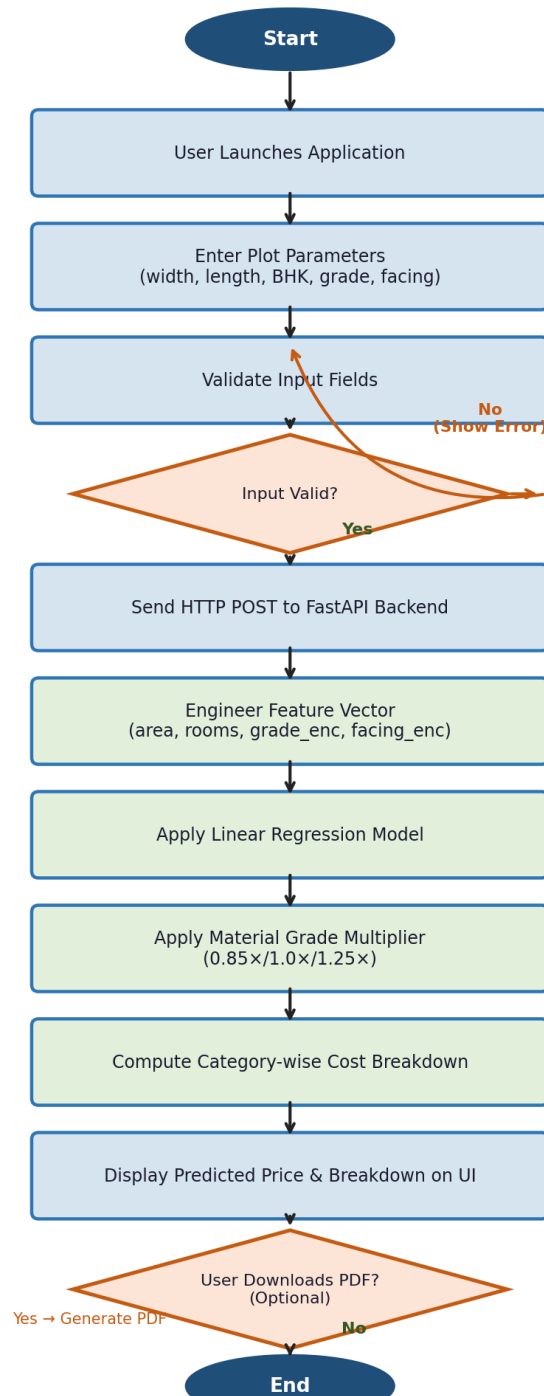


Figure 3.3 – Activity Diagram – Price Prediction System Workflow

3.3.4 Flowchart

The system flowchart represents all decision points in the processing pipeline. Key decision nodes: 'Is input data valid?' (if No: show error message; if Yes: proceed to API call), 'Is prediction successful?' (if No: show error; if Yes: display result), and 'User requests PDF export?' (if Yes: generate and download PDF; if No: end).

3.3.5 ER diagram

The Entity-Relationship diagram defines the logical schema of the database. Key entities and their attributes:

Entity	Primary Key	Key Attributes
User	User_ID (PK)	Username, Email, Password, Role
Prediction	Prediction_ID (PK)	Input_Data, Predicted_Price, Material_Grade, Date
ML_Model	Model_ID (PK)	Model_Name, Algorithm, Version, Trained_At
Feature	Feature_ID (PK)	Feature_Name, Feature_Value, Prediction_ID (FK)

Key Relationships: User creates Prediction; Prediction uses ML_Model; Prediction generates Feature records.

ER Diagram - AI Price Prediction System Database

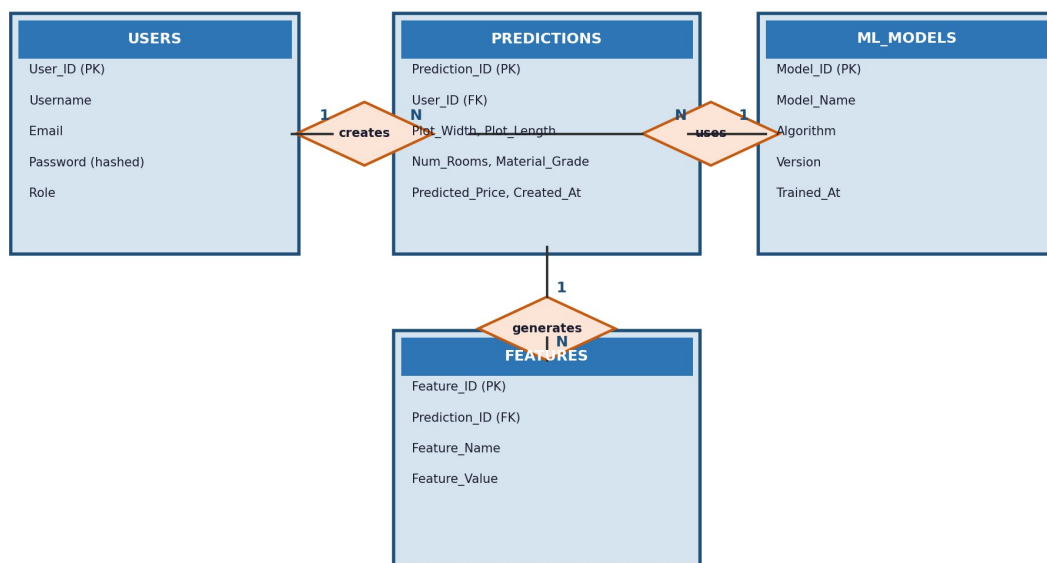


Figure 3.4 – ER Diagram – AI Price Prediction System Database

3.3.6 Sequence Diagram

The sequence diagram shows the time-ordered interaction between system components during a price prediction request:

Step	From	To	Message
1	User	PySide6 Controller	Enter plot dimensions, rooms, grade, direction
2	PySide6 Controller	PySide6 Controller	Collect and validate user input
3	PySide6 Controller	FastAPI Backend	Predict Price Request (HTTP POST with JSON)
4	FastAPI Backend	ML Model	Engineer features and run Linear Regression
5	ML Model	FastAPI Backend	Return predicted price + cost breakdown (JSON)

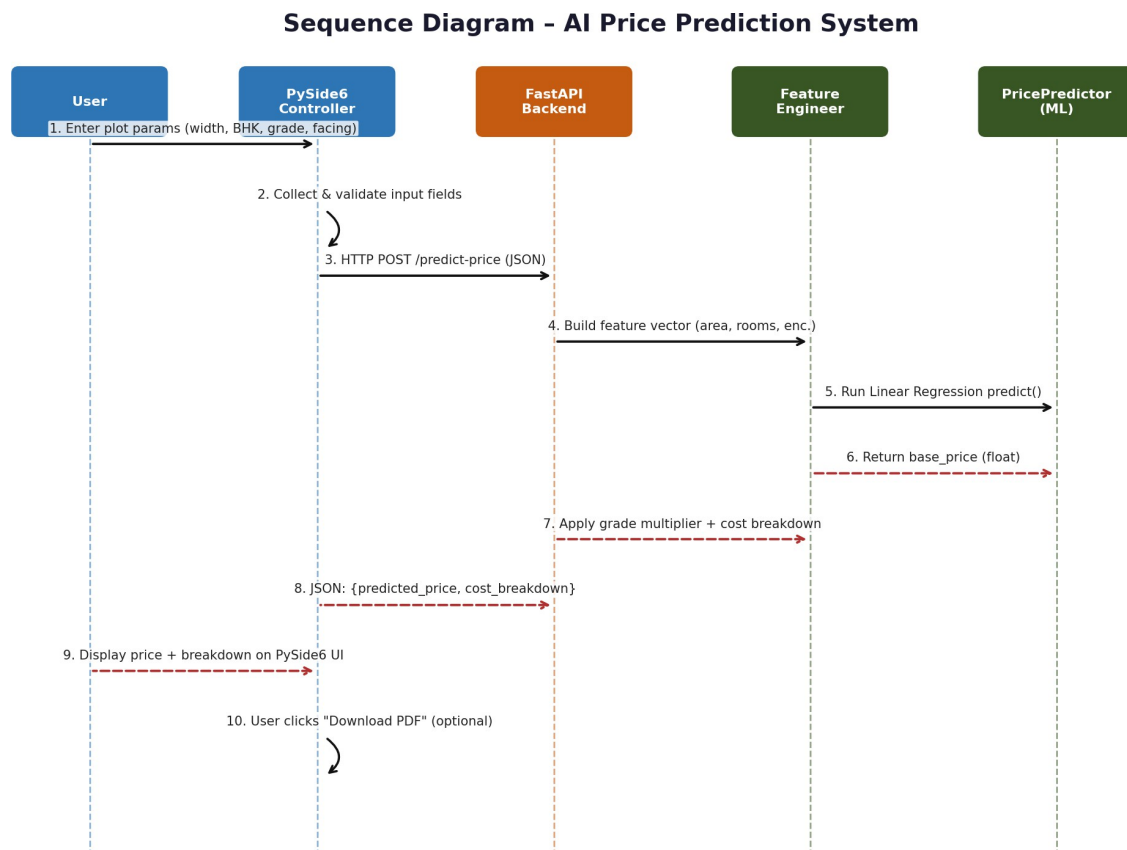


Figure 3.6 – Sequence Diagram – AI Price Prediction System

Step	From	To	Message
6	FastAPI Backend	PySide6 Controller	Send JSON results to Controller
7	PySide6 Controller	User	Display predicted price and breakdown on UI

4. Database Design

1. Logical Database Design

The logical design consists of four normalized tables: Users, Predictions, ML_Models, and Features. All relationships are properly enforced through foreign key constraints.

Field Name	Table	Type	Description
User_ID	Users	Integer (PK)	Unique user identifier
Username	Users	String	Login username
Email	Users	String	User email address
Password	Users	String (hashed)	Encrypted password
Role	Users	String	User role (admin/user)
Prediction_ID	Predictions	Integer (PK)	Unique prediction identifier
User_ID	Predictions	Integer (FK)	References Users table
Plot_Width	Predictions	Float	Plot width in feet
Plot_Length	Predictions	Float	Plot length in feet
Num_Rooms	Predictions	Integer	Number of BHK rooms
Material_Grade	Predictions	String	Economy / Standard / Premium
Predicted_Price	Predictions	Float	Estimated price in INR
Created_At	Predictions	Date	Timestamp of prediction
Model_ID	ML_Models	Integer (PK)	ML model identifier
Model_Name	ML_Models	String	e.g. LinearRegression_v1
Algorithm	ML_Models	String	e.g. Linear Regression
Feature_ID	Features	Integer (PK)	Feature record identifier
Feature_Name	Features	String	e.g. built_up_area, num_rooms
Feature_Value	Features	Float	Numeric feature value

3.4.2 Physical Database Design

The physical implementation uses a lightweight local relational database with easy migration to PostgreSQL or MySQL for cloud deployment. Table creation scripts use appropriate data

types: INTEGER for identifiers, TEXT for string fields, REAL for monetary values, and DATE for temporal fields.

Normal Form	Action Applied
1NF	All fields contain atomic values. No repeating groups in any table.
2NF	Separate tables: Users, Predictions, ML_Models, Features. No partial dependencies.
3NF	Transitive dependencies removed. Model details stored in ML_Models, not in Predictions.

Chapter 4: Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies

Tool / Technology	Version	Purpose
Python	3.10+	Core programming language for all system components
PySide6 / Qt6	6.x	Desktop GUI framework for the interactive user interface
FastAPI	0.100+	REST API backend for ML model serving
Scikit-learn	1.3+	Linear Regression model training and prediction
NumPy	1.24+	Numerical operations and feature vector construction
Pandas	2.0+	Data loading, cleaning, and feature engineering
ReportLab	4.0+	PDF generation for price report export
Uvicorn	0.20+	ASGI server for running the FastAPI backend
pickle	(stdlib)	ML model serialization and deserialization
VS Code / PyCharm	Latest	Integrated Development Environment
Git / GitHub	–	Version control and collaboration
Postman	Latest	API testing tool for FastAPI endpoint validation

1. PySide6 Desktop Frontend

PySide6 serves as the desktop GUI framework. The UI is organized into an input panel on the left and a result display area on the right. Qt Widgets are used for input fields (QLineEdit, QSpinBox, QComboBox for material grade) and buttons (QPushButton). A QTabWidget provides separate tabs for the Prediction Result and Cost Summary views. The price prediction output is displayed using a QLabel for the total price and a QTableWidgetItem for the cost breakdown.

2. FastAPI Backend

FastAPI is used to build the REST API backend server running locally on localhost:8000 using the Uvicorn ASGI server. The server exposes three endpoints:

Endpoint	Method	Input	Output
/predict-price	POST	feature JSON (plot_width, plot_length, num_rooms, material_grade, facing)	predicted_price (INR), cost_breakdown, material_grade
/history	GET	user_id (optional)	List of past prediction records
/health	GET	None	API status: 'running', version

4.1.3 ML Price Prediction Component

The PricePrediction Model uses a trained Linear Regression model to predict property price. The model is trained on a dataset of Indian residential real estate listings. Input features include total built-up area (sq. ft.), number of rooms, material grade (encoded as 0/1/2), and facing direction (encoded as 0/1/2/3). The model is trained offline using Scikit-learn, evaluated on a held-out test set, and serialized using Python's pickle module for runtime loading by the FastAPI backend.

Material grade cost multipliers are applied post-prediction: Economy grade applies a $0.85\times$ multiplier, Standard grade applies $1.0\times$, and Premium grade applies a $1.25\times$ multiplier. The category-wise cost breakdown is computed as fixed percentage splits of the total predicted price: structure (45%), finishing (25%), plumbing (15%), and electrical (15%).

4.2 Testing Techniques and Test Plans

Test Type	Description	Tools Used
Unit Testing	Individual functions for input validation, feature engineering, cost multiplier application, and PDF generation tested with sample inputs and boundary cases.	pytest
Integration Testing	Verified that PySide6 Controller correctly sends POST requests to FastAPI and receives valid price prediction JSON responses.	Postman, pytest
ML Model Testing	Validated Linear Regression model predictions against a held-out test dataset. Model verified for $R^2 > 0.80$ and acceptable MAE (Mean Absolute Error).	Scikit-learn metrics
UI Testing	Manually tested all UI components including input fields, dropdowns, Predict button, tab switching, and PDF download.	Manual Testing
API Endpoint Testing	Tested all FastAPI endpoints with valid inputs, missing fields, and invalid data types to verify correct responses and error handling.	Postman
PDF Export Testing	Verified PDF output contains correct prediction summary and cost breakdown table across various input configurations.	Manual Testing
User Acceptance Testing	System demonstrated to project guides and reviewed for accurate price estimation and intuitive UI.	Manual Review

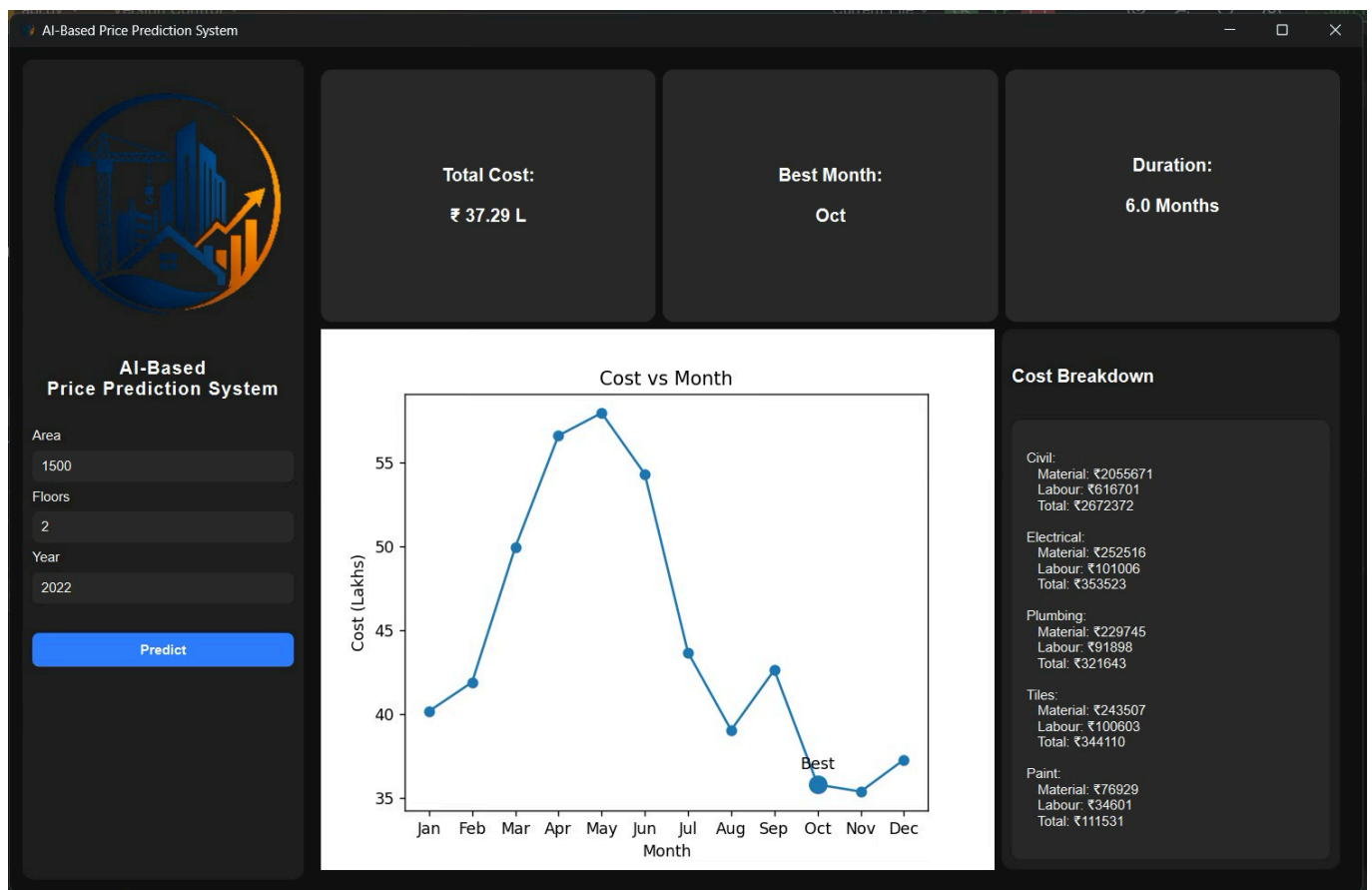
4.3 Installation Instructions

1. Install Python 3.10+ from python.org. Verify installation: `python --version`
2. Clone the project repository: `git clone https://github.com/[repo]/ai-price-prediction.git`
3. Navigate to the project directory: `cd ai-price-prediction`

4. Install required Python libraries: `pip install pyside6 fastapi uvicorn scikit-learn numpy pandas reportlab`
5. Navigate to the backend directory and start the FastAPI server: `cd backend && uvicorn main:app --reload --port 8000`
6. In a new terminal, navigate to the frontend directory and launch the PySide6 application: `cd frontend && python main.py`
7. The desktop application window will open. Enter property parameters and click 'Predict Price'.

4.4 End User Instructions

1. Launch the application by running: `python main.py` from the frontend directory.
2. In the Input Panel, enter the Plot Width (feet) and Plot Length (feet) in the respective fields.
3. Select the number of BHK rooms (1BHK to 5BHK) using the Room Count spinner.
4. Select the plot facing direction (North / South / East / West) from the dropdown.
5. Select the Material Grade (Economy / Standard / Premium) for cost estimation.
6. Click the 'Predict Price' button to process your input and display the predicted price within a few seconds.
7. Use the tabs to switch between the Prediction Result view and the Cost Summary view.
8. Click 'Download PDF' to export the complete price report as a PDF file.



Chapter 5: Results and Discussions

1. User Interface Representation

The PySide6 desktop application serves as the primary user interface of this system, organized into the following major sections:

Main Application Window: Divided into an Input Panel on the left and a Result Display Area on the right. The input panel contains all input fields, dropdowns, and the Predict Price button. The result area shows the predicted price, cost breakdown, and tabs for the summary view.

Price Prediction Display: The Prediction Result tab displays the total predicted property price in INR, prominently formatted with the material grade applied. A cost breakdown table shows structure, finishing, plumbing, and electrical costs as both absolute values and percentages.

Cost Summary View: The Cost Summary tab provides a visual representation of the cost distribution across construction categories, helping users understand how the total price is composed.

[Figure 5.1: Main Application Window – Input Panel – to be inserted here] [Figure 5.2: Price Result Display – Cost Breakdown View – to be inserted here] [Figure 5.3: PDF Export – Price Report – to be inserted here]

2. Brief Description of Various Modules

Module	Responsibility
UserInterface (PySideUI)	Base class providing initializeUI() and displayResult() interface for the PySide6 desktop application
InputHandler	Collects plot dimensions, room count, facing direction, and material grade from UI widgets
ResultDisplay	Renders the predicted price and cost breakdown table in the PySide6 UI tabs
PriceEstimator	Loads the trained Linear Regression model, engineers features, applies multipliers, computes cost breakdown
FastAPIServer	Receives HTTP POST requests, routes to the ML model, and returns JSON prediction results
FastAPIClient	Sends HTTP POST requests from the PySide6 Controller to the FastAPI Backend server
Controller	Orchestrates UI input collection → API request → response handling → result display pipeline
PDF Exporter	Generates downloadable PDF price reports combining prediction summary and cost breakdown table

3. Snapshots of System with Brief Details

Input Panel: Shows all input fields including Plot Width, Plot Length, Room Count (BHK), Facing Direction dropdown, and Material Grade selector. The Predict Price button is prominently displayed at the bottom of the input panel.

Price Prediction Output: Displays the total predicted property price in INR in large formatted text. Below it, a breakdown table shows: Structure Cost (45%), Finishing Cost (25%), Plumbing Cost (15%), Electrical Cost (15%), and Total Price. The applied material grade multiplier is also shown.

PDF Report: The exported PDF contains a header with project details, a summary section showing all input parameters, the predicted price, the material grade applied, and a formatted cost breakdown table.

[Figure 5.4: Database – Users Table – to be inserted here] [Figure 5.5: Database – Predictions Table – to be inserted here]

4. Back-End Representation

The back-end of this platform consists of: (1) A FastAPI REST server running on localhost:8000 that handles all ML model invocation and feature engineering; (2) The PricePrediction ML component using a trained Scikit-learn Linear Regression model loaded at startup via pickle; and (3) A local relational database for storing user input records and prediction history.

The FastAPI server is started independently before the PySide6 frontend and runs as a background process. The PySide6 Controller communicates with the FastAPI backend through an HTTP client class that wraps the Python requests library.

5. Snapshots of Database Tables with Brief Description

Table	Fields	Description
Users	User_ID (PK), Username, Email, Password, Role	Stores registered user credentials and roles for multi-user support
Predictions	Prediction_ID (PK), User_ID (FK), Plot_Width, Plot_Length, Num_Rooms, Material_Grade, Predicted_Price, Created_At	Stores all price prediction requests and results with full input parameters
ML_Models	Model_ID (PK), Model_Name, Algorithm, Version, Trained_At	Stores metadata about deployed ML model versions and training details
Features	Feature_ID (PK), Prediction_ID (FK), Feature_Name, Feature_Value	Stores individual engineered feature values used for each price prediction

Chapter 6: Summary and Conclusions

This project successfully designed and developed an AI-Based Price Prediction System — an intelligent, offline-capable desktop application that applies machine learning-based Linear Regression to deliver accurate real estate price estimates from structured user inputs. The system addresses a critical gap in the Indian real estate domain by delivering data-driven, expert-level property valuations to non-technical users instantly and without professional consultation.

The work carried out in this project can be summarized as follows:

1. A clean two-tier system architecture was designed and implemented, comprising the PySide6 desktop frontend and the FastAPI REST backend with an integrated Linear Regression price prediction component.
2. The PricePrediction Model was successfully trained using Linear Regression on Indian residential real estate data, producing interpretable price estimates based on plot area, room count, material grade, and facing direction.
3. Material grade cost multipliers (Economy 0.85×, Standard 1.0×, Premium 1.25×) were implemented to adjust base predictions according to construction quality, improving practical accuracy.
4. A detailed category-wise cost breakdown (structure, finishing, plumbing, electrical) was implemented, providing users with actionable insight into how the total price is composed.
5. The PySide6 desktop application provides an intuitive and responsive user interface with tabbed views for prediction results and cost summaries, and PDF export functionality.
6. The system operates fully offline, making it accessible to users in field conditions and in Tier-2/Tier-3 cities where internet connectivity may be unreliable.

Key conclusions drawn from the development and testing of this system include:

1. Linear Regression provides a transparent and interpretable model for property price prediction, where the contribution of each feature (area, rooms, grade) to the final price can be directly understood from the model coefficients — an important property for building user trust in the predictions.
2. The two-tier modular architecture effectively separates UI and ML concerns, enabling independent testing, updating, and scaling of each component without affecting the others.
3. The material grade multiplier mechanism significantly improves the practical relevance of predictions, as construction quality is one of the largest determinants of property value in the Indian market.
4. The system reduces the time required to obtain a property price estimate from several hours (when relying on expert consultation) to a matter of seconds, demonstrating the practical value of ML automation in this domain.
5. The PDF export feature makes the system's output immediately usable as a deliverable for client presentations, loan applications, and initial construction planning.

In conclusion, this project demonstrates that a well-designed ML system built on accessible open-source tools (Python, PySide6, FastAPI, Scikit-learn) can deliver professional-grade real estate valuation capabilities, making it highly relevant for homeowners, small developers, and architects in the Indian market.

Chapter 7: Future Scope

While the current platform successfully addresses core price prediction requirements, several enhancements are identified for future development:

Advanced ML Models: Upgrading from Linear Regression to more powerful models such as Gradient Boosting (XGBoost/LightGBM) or Neural Networks to improve prediction accuracy, especially for non-linear relationships between property features and market price.

Cloud-Based Deployment: Migrating the FastAPI backend to a cloud platform (AWS / Azure / Render) and developing a web browser-based frontend using React or Vue.js, enabling access from any device without local installation.

Mobile Application: Developing a mobile version of the application for Android and iOS using Flutter or React Native, allowing homeowners and developers to obtain price estimates directly from their smartphones.

Multi-City Price Prediction: Expanding the price prediction model to support location-specific pricing across multiple Indian cities by integrating geo-tagged real estate datasets and city-level cost multipliers, enabling accurate estimates for Delhi, Mumbai, Bangalore, and other major markets.

Real-Time Market Data Integration: Integrating with real estate listing APIs (e.g., 99acres, MagicBricks) to dynamically update the training dataset with current market prices, ensuring the model remains accurate as market conditions change.

Natural Language Input: Adding support for natural language property descriptions (e.g., '3BHK flat, 1200 sqft, East facing, premium finish') using NLP parsing to automatically extract features and generate a price prediction.

Comparative Market Analysis: Adding a feature that displays the predicted price alongside comparable recent property transactions in the same area, providing context for the estimate and improving user confidence.

User Dashboard and History: Implementing a full user authentication system with a personal dashboard where users can view their prediction history, compare estimates over time, and track market price trends.

Explainability (XAI): Integrating SHAP (SHapley Additive exPlanations) values to provide feature-level explanations for each prediction, helping users understand why a specific price was predicted and which factors contributed most.

API-as-a-Service: Exposing the price prediction API as a commercial service with API key authentication, rate limiting, and usage analytics, allowing third-party real estate platforms to integrate the prediction capability into their own applications.

These enhancements would progressively transform the current desktop prototype into a production-ready, enterprise-grade AI price prediction platform suitable for large-scale real estate applications, government housing initiatives, and financial institutions.

Appendix

Appendix I – Sample Python Code: Model Training

The following Python code illustrates the Linear Regression model training and serialization:

```
import pandas as pd
import numpy as np
import pickle
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error

# Load and prepare dataset
df = pd.read_csv('real_estate_data.csv')
df['built_up_area'] = df['plot_width'] * df['plot_length'] * 0.6
df['grade_enc'] =
df['material_grade'].map({'Economy':0, 'Standard':1, 'Premium':2})
df['facing_enc'] =
df['facing'].map({'North':0, 'South':1, 'East':2, 'West':3})

X = df[['built_up_area', 'num_rooms', 'grade_enc', 'facing_enc']]
y = df['price_inr']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(f'R2 Score: {r2_score(y_test, y_pred):.3f}')
print(f'MAE: {mean_absolute_error(y_test, y_pred):.0f} INR')

with open('price_model.pkl', 'wb') as f:
    pickle.dump(model, f)
```

Appendix II – Sample Python Code: Price Prediction

The following Python code illustrates the prediction and cost breakdown computation:

```
import pickle
import numpy as np
with open('price_model.pkl', 'rb') as f: model = pickle.load(f)

GRADE_MULTIPLIERS = {'Economy': 0.85, 'Standard': 1.0, 'Premium': 1.25}

COST_SPLIT = {'Structure': 0.45, 'Finishing': 0.25,
              'Plumbing': 0.15, 'Electrical': 0.15}

def predict_price(plot_width, plot_length, num_rooms, material_grade,
facing):
    built_up_area = plot_width * plot_length * 0.6
    grade_enc = {'Economy':0, 'Standard':1, 'Premium':2}[material_grade]
    facing_enc = {'North':0, 'South':1, 'East':2, 'West':3}[facing]
```

```

    features = np.array([[built_up_area, num_rooms, grade_enc,
facing_enc]])
    base_price = model.predict(features)[0]
    final_price = base_price * GRADE_MULTIPLIERS[material_grade]
    breakdown = {k: round(final_price * v, 2) for k, v in
COST_SPLIT.items()}
    return {'predicted_price': round(final_price, 2), 'breakdown':
breakdown}

```

Appendix III – FastAPI Endpoint Specification

Endpoint	Method	Input	Output
/predict-price	POST	JSON: plot_width, plot_length, num_rooms, material_grade, facing	predicted_price (INR), cost_breakdown, grade_multiplier
/history	GET	Optional: user_id query param	List of past prediction records
/health	GET	None	API status: 'running', version, model_loaded: true

Bibliography

1. A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O'Reilly Media, 2022.
2. D. Harrison and D. L. Rubinfeld, "Hedonic Housing Prices and the Demand for Clean Air," Journal of Environmental Economics and Management, vol. 5, no. 1, pp. 81–102, 1978.
3. FastAPI Official Documentation, <https://fastapi.tiangolo.com/tutorial/>, accessed on 10/03/2026.
4. PySide6 / Qt for Python Official Documentation, <https://doc.qt.io/qtforpython-6/>, accessed on 12/03/2026.
5. Scikit-learn: Machine Learning in Python, <https://scikit-learn.org>, accessed on 15/03/2026.
6. IEEE Software Documentation Standard, IEEE Std 830-1998, IEEE Computer Society, 1998.
7. Bureau of Indian Standards, "National Building Code of India 2016," BIS, New Delhi, 2016.
8. Construction Industry Development Council (CIDC), "Construction Cost Data for Residential Buildings in India," CIDC, New Delhi, 2023.
9. ReportLab PDF Generation Library Documentation, <https://www.reportlab.com/docs/>, accessed on 22/03/2026.
10. NumPy Developers, "NumPy Reference Documentation," 2023. Available: <https://numpy.org/doc/>